

GenAI Project Assessment:

Prompt & Metrics Report

Student Name: Ashmita Reddy

Date: 5th June, 2026

Project Chosen: Project Lens

AI Assistant Used: Antigravity and Gemini

1. Executive Summary & Final Metrics

- **Total Prompts Issued:** 2
- **Prompts Yielding Functional Code (No edits needed):** NA
- **Total Lines of Code (LOC) in Final Project:** 720 lines of code
- **Total Lines of Code Manually Edited/Written by Human:** 4 lines

Final Scores

- **AI Accuracy Rate:** [Calculated %] (Formula: [Functional Prompts / Total Prompts] x 100) - $1 / 2 = 0.5 \times 100 = 50\%$ is the accuracy rate
- **Human Correction Rate:** [Calculated %] (Formula: [Manually Edited Lines / Total Project Lines] x 100) - $4 / 720 = 0.0055555556 \times 100 = 0.55555556$ is the human accuracy rate.

2. Master Prompt & Modification Log

Log every single interaction with your AI assistant here. Be brutal and honest about where the AI failed or hallucinated.

Prompt	Exact Prompt Text Used	Status	Action Taken / Changes Made
--------	------------------------	--------	-----------------------------

01	Gemini - Refine the prompt in order to provide to antigravity, the prompt should consist of 1. Persona, Project working, features, Instructutions, Input output examples, constraints “In the competitive engineering placement landscape, freshers often struggle to understand if their academic and capstone projects are technically robust enough for specific high-paying domains like Fintech or Health-tech or more based on product or service companies as well. it is an advanced developer intelligence platform that bridges this gap. It acts as an automated technical auditor and mock-interviewer. the platform runs a hybrid vector search against real-world engineering standards and domain rubrics to determine a project's structural suitability. It identifies missing technical features, generates a step-by-step roadmap to scale the project, and spins up an interactive, domain-specific interview simulator to prepare students for rigorous technical rounds. Technical Stack • Frontend: React.js, Tailwind CSS (via CDN for rapid building), Axios for API requests. • Backend: Node.js, Express.js (Fast, lightweight async routing). • Database: ChromaDB from cloud • LLM Integration: Gemini API (Primary) + Groq API (Failover routing). UI The UI must have like a chat inference where in the center little bottom or in middle has domain dropdown, tech stack dropdown, role drop down side by side (all these must have certain options and other option as well where they can type it out, these fields input is must if the user doesnt fill it it should ask them to fill the necessary fields) below these, project description in a chat like type the ouput should geenrate in a split screen or below it but this should be visisble more it should have if it is sutibale or not box, evaluation of this particular model (the output accuracy)	Accepted	NA
----	--	----------	----

	Constraints: 1. DO NOT USE ports like 5000 here mac os configurations might occur 2. make sure you are using chroma db cloud import functions as im doing it from browser 3. .env should be created in backend root foler with variables of gemini api, groq api, chroma db related vars 4. DO NOT hard code ports as this needs to be deployed both backend and frontend together in vercel (a monorepo) 5. include data folder in backend where ill manually upload 1 pdf and please make sure these chunks and everything is ingested into the chromadb and runs even when im deploying it 6. make sure you include npm start and npm run dev in package.json		
--	---	--	--

2	Antigrvaity - You are a Senior Full-Stack Engineer, AI Systems Architect, and Product Designer with expertise in: • React.js • Node.js / Express.js • ChromaDB Cloud • Retrieval-Augmented Generation (RAG) • Gemini API • Groq API • Vercel Deployment • Secure AI Applications • Technical Interview Platforms • ATS & Engineering Hiring Systems Your goal is to build a production-ready application called Project-Lens. The final output must be a complete, deployable monorepo with frontend and backend integrated and ready for Vercel deployment. Do not generate pseudocode. Generate real implementation-ready code, folder structures, environment setup, API routes, ingestion scripts, deployment configuration, and documentation. PROJECT NAME Project-Lens PROJECT DESCRIPTION Project-Lens is an AI-powered Developer Intelligence Platform that helps engineering students determine whether their projects are strong enough for specific technical domains and job roles. Many students build projects without understanding whether they meet the expectations of modern engineering teams. Project-Lens acts as: 1. Technical Project Auditor 2. Domain Suitability Evaluator 3. Project Scaling Advisor 4. Mock Technical Interviewer The platform compares a student's project against real-world engineering standards stored inside ChromaDB and provides actionable feedback. The goal is to answer: • Is this project suitable for my target role? • Is this project suitable for my target industry? • What advanced features are missing?	Accepted	NA
---	---	----------	----

CORE FEATURES 1. Hybrid Domain Suitability Auditor Inputs Target Domain (Required) Dropdown options: FinTech HealthTech EdTech E-Commerce SaaS Cybersecurity AI/ML Cloud Engineering DevOps Data Engineering Enterprise Software Product-Based Companies Service-Based Companies Other If "Other" selected: Show custom input field. Target Role (Required) Dropdown options: Frontend Developer Backend Developer Full Stack Developer Software Engineer Data Analyst Data Engineer Machine Learning Engineer AI Engineer DevOps Engineer Cloud Engineer Security Engineer Product Engineer Other If "Other" selected: Show custom input field. Tech Stack (Required) Java full stack Chroma db Gemini and Groq api Constraints DO NOT hardcode ports. DO NOT assume localhost ports. Use environment variables. Frontend and backend must work in a monorepo deployment. Create: vercel.json		
--	--	--

3. Visual Evidence & Code Artifacts

Project-Lens

+ New Analysis

HISTORY

SaaS - Full Stack Developer
Score: 85 • 05/06/2026

E-Commerce - Full Stack D...
Score: 65 • 04/06/2026

E-Commerce - Full Stack D...
Score: 35 • 04/06/2026

65

✓ Moderately Suitable

This project provides a solid foundation for a high-throughput shopping cart API, leveraging modern technologies like FastAPI, Docker, Redis, and PostgreSQL. While it addresses basic caching needs, it currently lacks several advanced architectural patterns and production-grade features essential for enterprise-level e-commerce systems handling extreme traffic volatility and complex operational demands.

✓ Strengths

- Utilizes a modern, performant tech stack (Python, FastAPI, Docker).
- Incorporates Redis for caching, indicating an awareness of performance optimization.
- PostgreSQL for order management is a robust choice for transactional data.
- Containerization with Docker facilitates deployment, scalability, and environment consistency.

✗ Weaknesses

- Lacks explicit implementation of Command Query Responsibility Segregation (CQRS) which is critical for high-throughput e-commerce.
- Does not mention a distributed search engine (e.g., Elasticsearch/Solr) for complex product catalog capabilities.
- The caching strategy described does not detail multi-tiered caching or sophisticated distributed cache invalidation mechanisms.
- Absence of message queues for asynchronous processing, which is vital for handling write spikes and event-driven architectures.
- No mention of an observability stack (logging, metrics, tracing) for production monitoring and debugging.